

Chameleon: a Heterogeneous and Disaggregated Accelerator System for Retrieval-Augmented Language Models

Wenqi Jiang
Systems Group, ETH Zurich
wenqi.jiang@inf.ethz.ch

Marco Zeller
Systems Group, ETH Zurich
mzeller@student.ethz.ch

Roger Waleffe
University of Wisconsin Madison
waleffe@wisc.edu

Torsten Hoefer
SPCL, ETH Zurich
torsten.hoefer@inf.ethz.ch

Gustavo Alonso
Systems Group, ETH Zurich
alonso@inf.ethz.ch

ABSTRACT

A Retrieval-Augmented Language Model (RALM) combines a large language model (LLM) with a vector database to retrieve context-specific knowledge during text generation. This strategy facilitates impressive generation quality even with smaller models, thus reducing computational demands by orders of magnitude. To serve RALMs efficiently and flexibly, we propose *Chameleon*, a heterogeneous accelerator system integrating both LLM and vector search accelerators in a disaggregated architecture. The *heterogeneity* ensures efficient serving for both inference and retrieval, while the *disaggregation* allows independent scaling of LLM and vector search accelerators to fulfill diverse RALM requirements. Our Chameleon prototype implements vector search accelerators on FPGAs and assigns LLM inference to GPUs, with CPUs as cluster coordinators. Evaluated on various RALMs, Chameleon exhibits up to 2.16× reduction in latency and 3.18× speedup in throughput compared to the hybrid CPU-GPU architecture. The promising results pave the way for adopting heterogeneous accelerators for not only LLM inference but also vector search in future RALM systems.

PVLDB Reference Format:

Wenqi Jiang, Marco Zeller, Roger Waleffe, Torsten Hoefer, and Gustavo Alonso. Chameleon: a Heterogeneous and Disaggregated Accelerator System for Retrieval-Augmented Language Models. PVLDB, 18(1): 42 – 52, 2024.

doi:10.14778/3696435.3696439

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/fpgasystems/Chameleon-RAG-Acceleration>.

1 INTRODUCTION

Vector databases facilitate the development of *Retrieval-Augmented Language Models (RALMs)*, an increasingly popular approach to serve generative large language models (LLMs). The architecture of RALM, as shown in Figure 1, allows the LLM to focus on learning linguistic structures, while incorporating context-specific knowledge during inference. Specifically, the external textual knowledge

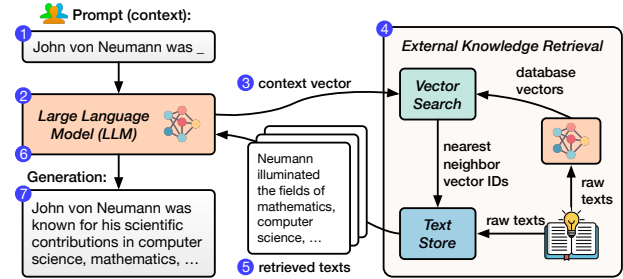


Figure 1: A retrieval-augmented language model (RALM).

is encoded as vectors using LLMs and stored in a vector database. Given an inference context (e.g., a prompt), the knowledge retriever identifies relevant knowledge in the database via vector search, which assesses relevance by computing the similarity between the context vector and the database vectors. The retrieved texts are then incorporated into the LLM to facilitate high-quality generation.

RALMs show three major advantages over conventional LLMs. *First of all*, RALMs, even using smaller LLMs with one to two orders of magnitude fewer parameters, can match or surpass the generation quality of conventional LLMs on various tasks [23, 31, 41–43, 48, 49], thus significantly lowering the inference cost. This is because conventional LLMs rely on a vast number of parameters trained on massive datasets to capture and retain textual knowledge [8, 12, 67, 74], while RALMs can integrate retrieved knowledge during inference, not burdening the LLM’s parameters. *Moreover*, knowledge editing in RALMs is as straightforward as updating the database, enabling efficient integration of new or private knowledge [3, 4]. In contrast, updating knowledge in conventional LLMs is inflexible, requiring additional training [7, 49]. *Finally*, RALMs enhance the reliability and interpretability of generated content by sourcing knowledge externally, while conventional LLMs are prone to producing non-factual content, known as hallucination [49, 50].

Despite its advantages, efficient RALM inference presents **two challenges**. *First, the workload characteristics of the LLM and the retriever are distinct*. While the LLM inference primarily relies on rapid tensor operations, the vector search system — often utilizing fast and memory-efficient search algorithms like Product Quantization (PQ) [35] — demands both substantial memory capacity to hold the vectors and fast processing of quantized database vectors during query time. *Second, the diverse range of RALM configurations leads to shifting system requirements and bottlenecks*. Regarding retrieval frequency, some models retrieve once per generated token [6, 42, 57], while others retrieve only once per entire sequence [31, 49]. In

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 18, No. 1 ISSN 2150-8097.
doi:10.14778/3696435.3696439